

Execution Plans in SQL Server with real-time examples using AdventureWorks2019.

Execution plans show **how SQL Server's Query Optimizer decides to execute a query** (which indexes to use, how joins happen, row estimates, etc.). There are **two types**:

- **Estimated Execution Plan** → Query optimizer's guess before running.
- **Actual Execution Plan** → What actually happened when query ran (with runtime stats).

Now let's go through **all major Execution Plan operators & scenarios** one by one with AdventureWorks examples.

◆ 1. Table Scan

👉 When no index is available, SQL Server scans the whole table.

Scenario: Querying a column without an index.

-- No index on MiddleName

```
SELECT MiddleName
FROM Person.Person
WHERE MiddleName = 'John';
```

- Execution Plan: **Table Scan** on Person.Person.
- Impact: Reads entire table → costly for large tables.

◆ 2. Clustered Index Scan

👉 Scans the entire clustered index (basically the table data itself).

Scenario:

```
SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person;
```

- Execution Plan: **Clustered Index Scan** on PK_Person_BusinessEntityID.
- Why: Query needs most/all rows, so scanning is cheaper than using an index seek.

◆ 3. Clustered Index Seek

👉 Uses clustered index to directly find rows.

Scenario:

```
SELECT *
FROM Person.Person
WHERE BusinessEntityID = 25;
```

- Execution Plan: **Clustered Index Seek** on PK_Person_BusinessEntityID.
- Efficient: Jumps directly to row with BusinessEntityID = 25.

◆ 4. Nonclustered Index Seek

👉 Uses nonclustered index for faster lookups.

Scenario:

```
-- Assume index exists on EmailAddress
SELECT EmailAddress
FROM Person.EmailAddress
WHERE EmailAddress = 'david@adventure-works.com';
```

- Execution Plan: **Index Seek** on nonclustered index IX_EmailAddress_EmailAddress.
- Efficient for selective searches.

◆ 5. Key Lookup (Bookmark Lookup)

👉 When nonclustered index provides partial data → SQL Server goes back to clustered index for rest.

Scenario:

```
SELECT BusinessEntityID, EmailAddress
```

```
FROM Person.EmailAddress
```

```
WHERE EmailAddress = 'david@adventure-works.com';
```

- Execution Plan: **Index Seek (Nonclustered) + Key Lookup** (Clustered).
- Fix: Use **covering index** → include needed columns.

```
CREATE NONCLUSTERED INDEX IX_EmailAddress_Covering
```

```
ON Person.EmailAddress (EmailAddress)
```

```
INCLUDE (BusinessEntityID);
```

◆ 6. RID Lookup

👉 Happens in **heap tables** (no clustered index).

Not in AdventureWorks since most tables are clustered.

But if we create a heap:

```
CREATE TABLE dbo.TestHeap (ID INT, Name NVARCHAR(50));
```

```
INSERT INTO dbo.TestHeap VALUES (1,'A'),(2,'B'),(3,'C');
```

-- With nonclustered index

```
CREATE NONCLUSTERED INDEX IX_TestHeap_ID ON dbo.TestHeap(ID);
```

```
SELECT Name FROM dbo.TestHeap WHERE ID = 2;
```

- Execution Plan: **Index Seek + RID Lookup**.

◆ 7. Nested Loops Join

👉 Good for **small data sets**.

Scenario:

```
SELECT p.FirstName, p.LastName, e.JobTitle
```

```
FROM Person.Person p
```

```
JOIN HumanResources.Employee e
```

```
ON p.BusinessEntityID = e.BusinessEntityID;
```

- Execution Plan: **Nested Loops Join** between Person and Employee.
- Efficient when one input is small.

◆ 8. Merge Join

👉 Best for **sorted inputs** (both sides indexed/sorted).

Scenario:

```
SELECT soh.SalesOrderID, sod.ProductID
```

```
FROM Sales.SalesOrderHeader soh
```

```
JOIN Sales.SalesOrderDetail sod
```

```
ON soh.SalesOrderID = sod.SalesOrderID;
```

- Execution Plan: **Merge Join** (since both inputs are ordered on SalesOrderID).

◆ 9. Hash Join

👉 Used for **large unsorted data sets**.

Scenario:

```
SELECT p.FirstName, p.LastName, s.SalesYTD
FROM Person.Person p
JOIN Sales.SalesPerson s
ON p.BusinessEntityID = s.BusinessEntityID;
```

- Execution Plan: **Hash Match (Inner Join)**.
- SQL builds hash table of smaller set → probes with larger set.

◆ 10. Sort Operator

👉 Needed when query requires ordered data.

Scenario:

```
SELECT FirstName, LastName
FROM Person.Person
ORDER BY LastName;
```

- Execution Plan: **Sort**.
- Expensive for large tables → can use indexed column to avoid extra sort.

◆ 11. Aggregate Operators

👉 Compute aggregates (SUM, COUNT, AVG, etc.).

Scenario:

```
SELECT SalesPersonID, SUM(TotalDue) AS TotalSales
FROM Sales.SalesOrderHeader
GROUP BY SalesPersonID;
```

- Execution Plan: **Stream Aggregate** (if input is sorted).
- Or **Hash Aggregate** (if unsorted).

◆ 12. Parallelism Operators

👉 SQL Server splits query across CPUs.

Scenario:

```
SELECT COUNT(*)
FROM Sales.SalesOrderDetail;
```

- On large tables, you may see **Parallelism (Distribute Streams, Repartition Streams, Gather Streams)**.
- Parallel plans improve performance for big datasets.

◆ 13. Index Spool / Table Spool

👉 Temporary worktable in tempdb.

Scenario:

```
SELECT p.BusinessEntityID, p.FirstName, e.JobTitle
FROM Person.Person p
JOIN HumanResources.Employee e
ON p.BusinessEntityID = e.BusinessEntityID
WHERE EXISTS (SELECT 1 FROM Sales.SalesOrderHeader soh
              WHERE soh.SalesPersonID = e.BusinessEntityID);
```

- Execution Plan: **Table Spool (Lazy Spool)**.
- Indicates optimizer reuses intermediate result sets.

◆ 14. Bitmap Filter

👉 Seen in large parallel plans, often with hash joins.
Used to filter rows efficiently before join.

◆ 15. Remote Query Operators

👉 Seen if linked servers or external queries are used.

✅ Summary – Execution Plan Analysis Path

- **Scans vs Seeks** → Efficiency depends on indexes.
- **Lookups** → Key Lookup or RID Lookup = performance warning.
- **Joins** → Nested Loops (small), Merge (sorted), Hash (big).
- **Extra Ops** → Sort, Spool, Parallelism = red flags for tuning.

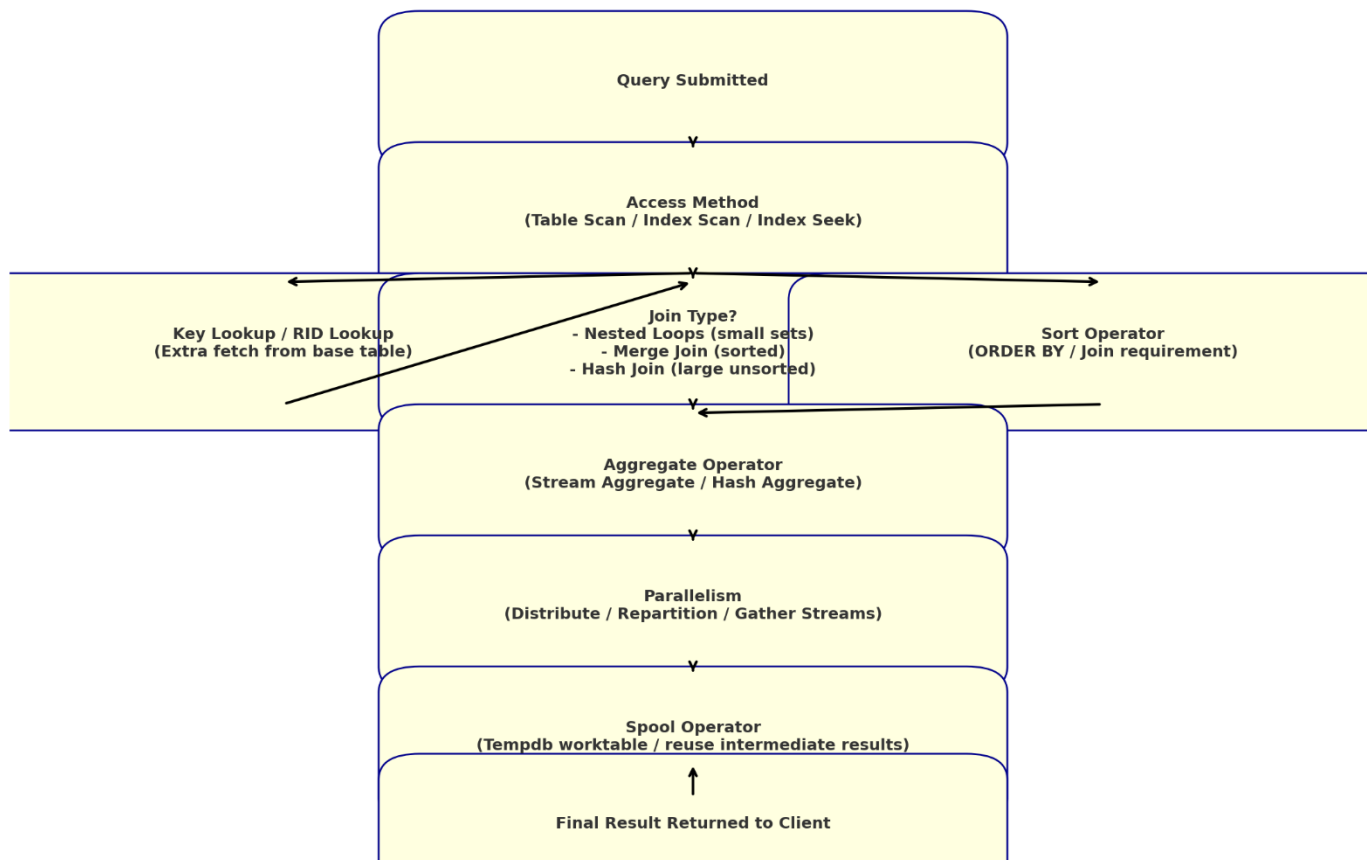
⚡ Real-time DBA Tip:

When troubleshooting, always look at:

- **Estimated vs Actual rows** (if mismatch → bad statistics).
- **Expensive operators** (CPU, IO heavy).
- **Warnings** (missing indexes, spills to tempdb).

Here's the **Execution Plan Operators Flowchart**  showing how SQL Server moves from **Access Methods (Scan/Seek)** → **Lookups** → **Joins** → **Sort** → **Aggregates** → **Parallelism/Spool** → **Final Result**.

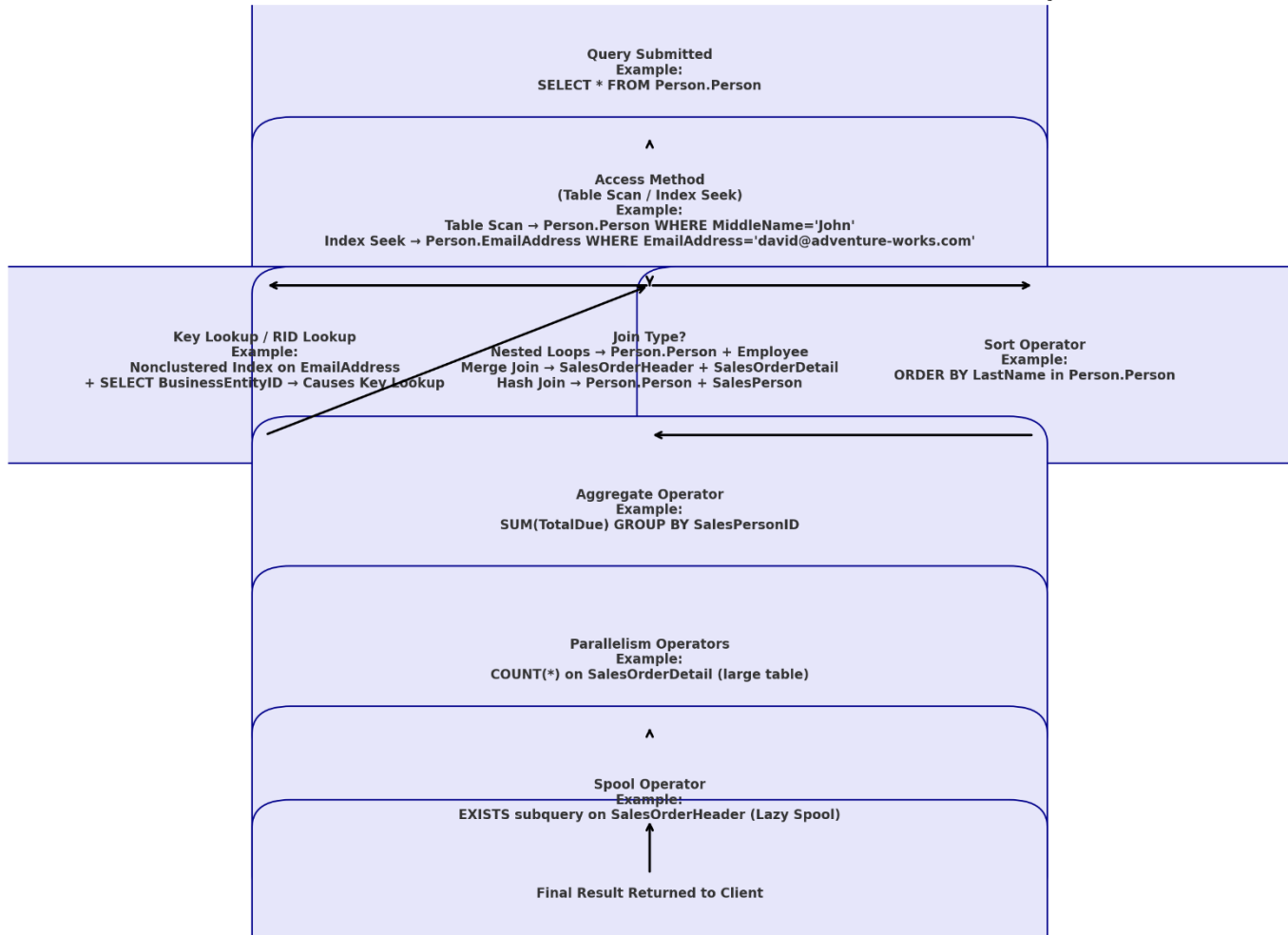
SQL Server Execution Plan Operators Flowchart



Here's the **Execution Plan Flowchart with real AdventureWorks2019 query examples**

so you can directly map each operator (Scan, Seek, Lookup, Join, Sort, Aggregate, Parallelism, Spool) with a real query demo.

SQL Server Execution Plan Flowchart with AdventureWorks Examples



```
/* =====
SQL Server Execution Plan Operators Demo
Database: AdventureWorks2019
Purpose: Demonstrate execution plan operators
===== */
```

-- 1. Table Scan (no index usage)

```
SELECT MiddleName
FROM Person.Person
WHERE MiddleName = 'John';
```

-- 2. Clustered Index Scan (PK is clustered)

```
SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person;
```

-- 3. Clustered Index Seek

```
SELECT *  
FROM Person.Person  
WHERE BusinessEntityID = 25;
```

-- 4. Nonclustered Index Seek

```
CREATE NONCLUSTERED INDEX IX_EmailAddress_EmailAddress  
ON Person.EmailAddress (EmailAddress);  
GO
```

```
SELECT EmailAddress  
FROM Person.EmailAddress  
WHERE EmailAddress = 'david@adventure-works.com';
```

-- 5. Key Lookup (when index doesn't cover query)

```
SELECT BusinessEntityID, EmailAddress  
FROM Person.EmailAddress  
WHERE EmailAddress = 'david@adventure-works.com';
```

-- Fix with covering index

```
CREATE NONCLUSTERED INDEX IX_EmailAddress_Covering  
ON Person.EmailAddress (EmailAddress)  
INCLUDE (BusinessEntityID);
```

-- 6. RID Lookup (Heap table example)

```
CREATE TABLE dbo.TestHeap (ID INT, Name NVARCHAR(50));  
INSERT INTO dbo.TestHeap VALUES (1,'A'),(2,'B'),(3,'C');
```

```
CREATE NONCLUSTERED INDEX IX_TestHeap_ID ON dbo.TestHeap(ID);  
GO
```

```
SELECT Name FROM dbo.TestHeap WHERE ID = 2;
```

-- 7. Nested Loops Join

```
SELECT p.FirstName, p.LastName, e.JobTitle  
FROM Person.Person p  
JOIN HumanResources.Employee e  
ON p.BusinessEntityID = e.BusinessEntityID;
```

-- 8. Merge Join

```
SELECT soh.SalesOrderID, sod.ProductID  
FROM Sales.SalesOrderHeader soh  
JOIN Sales.SalesOrderDetail sod  
ON soh.SalesOrderID = sod.SalesOrderID;
```

-- 9. Hash Join

```
SELECT p.FirstName, p.LastName, s.SalesYTD
FROM Person.Person p
JOIN Sales.SalesPerson s
  ON p.BusinessEntityID = s.BusinessEntityID;
```

-- 10. Sort Operator

```
SELECT FirstName, LastName
FROM Person.Person
ORDER BY LastName;
```

-- 11. Aggregates

```
SELECT SalesPersonID, SUM(TotalDue) AS TotalSales
FROM Sales.SalesOrderHeader
GROUP BY SalesPersonID;
```

-- 12. Parallelism Operators

```
SELECT COUNT(*)
FROM Sales.SalesOrderDetail;
```

-- 13. Spool Operator

```
SELECT p.BusinessEntityID, p.FirstName, e.JobTitle
FROM Person.Person p
JOIN HumanResources.Employee e
  ON p.BusinessEntityID = e.BusinessEntityID
WHERE EXISTS (
  SELECT 1
  FROM Sales.SalesOrderHeader soh
  WHERE soh.SalesPersonID = e.BusinessEntityID
);
```

-- Clean up demo objects

```
DROP INDEX IX_EmailAddress_EmailAddress ON Person.EmailAddress;
DROP INDEX IX_EmailAddress_Covering ON Person.EmailAddress;
DROP TABLE dbo.TestHeap;
```